CSE102 Assignment 8

1. Overview

This report evaluates the C program submitted for CSE102 Assignment 8. The task involves reading integer data from a text file, processing it using mathematical operations (max/avg) over sliding windows (chunks), and generating color-coded output.

2. Function Descriptions

2.1 max_operation

This function takes two integer arrays of the same size and multiplies their elements pairwise. It then returns the maximum value from the resulting products.

Code snippet:

```
int max_operation(int* chunk, int* corresponding_chunk_part, int
chunk_size) {
   int max = chunk[0] * corresponding_chunk_part[0];
   for (int i = 1; i < chunk_size; i++) {
      int product = chunk[i] * corresponding_chunk_part[i];
      if (product > max) {
         max = product;
      }
   }
   return max;
}
```

2.2 avg_operation

This function computes the average of the products of two arrays by multiplying corresponding elements and dividing the total sum by the number of elements.

Code snippet:

```
int avg_operation(int* chunk, int* corresponding_chunk_part, int
chunk_size) {
   int sum = 0;
   for (int i = 0; i < chunk_size; i++) {
      sum += chunk[i] * corresponding_chunk_part[i];
   }
   return sum / chunk_size;
}
```

2.3 read_line

This function reads a single line from a file, removes the newline character, and tokenizes the string by commas to convert it into an integer array.

Code snippet:

```
int* read_line(FILE* file, int* length) {
  char buffer[300];
  if (file == NULL || fgets(buffer, sizeof(buffer), file) == NULL) {
    return NULL;
  }
  buffer[strcspn(buffer, "\n")] = '\0';

  int* arr = NULL;
  int count = 0;
  char* token = strtok(buffer, ",");
  while (token != NULL) {
    arr = realloc(arr, (count + 1) * sizeof(int));
    arr[count] = atoi(token);
    count++;
    token = strtok(NULL, ",");
  }
  *length = count;
  return arr;
}
```

2.4 process_line

This function processes a full data line using a sliding window approach. It applies the selected operation ('m' for max or 'a' for avg) to each chunk and stores the results in a new array.

Code snippet:

```
int* process_line(int* line, int line_length, int* multipliers, int chunk_size,
char operation) {
  int num_chunks = line_length - chunk_size + 1;
  int* results = malloc(num_chunks * sizeof(int));
  for (int i = 0; i < num_chunks; i++) {
    int* chunk = line + i;
    if (operation == 'm') {
      results[i] = max_operation(chunk, multipliers, chunk_size);
    } else {
      results[i] = avg_operation(chunk, multipliers, chunk_size);
    }
  }
```

```
    return results;
}
```

2.5 print_output_file()

This function processes each data line (R, G, B) using a sliding window, applying the chosen operation function (either max_operation or avg_operation) on each chunk of data.

```
int* print_output_file(int* line, int line_length, int* chunk, int chunk_size,
                       int (*operation)(int*, int*, int)) {
    int num_results = line_length - chunk_size + 1;
    int* results = malloc(num_results * sizeof(int));
    for (int i = 0; i < num_results; i++) {
        int* chunk_part = line + i;
        results[i] = operation(chunk_part, chunk, chunk_size);
    }
    return results;
}
```

3.Input/Output

The program reads data from 'input.txt'. The first line contains the weight vector (length must be 3, 6, or 9). The next three lines represent R, G, B channel values. Based on the user's choices, results are written to 'output.txt' in one of the selected color modes.

Users are prompted to enter three operations (max/avg) which are pushed onto the stack. They are also prompted to select a color mode (rgb, bgr, or gray).

Example contents of input.txt used for testing:
1,7,3,40,5,61,7,22,9

15,21,12,28,389,24,34,64,88,58,25,4,67,81,97,32,72,74,71,150

54,23,98,47,83,15,67,29,71,36,90,59,19,41,72,12,61,4,82,11

37,92,56,14,75,63,8,21,66,39,89,50,27,48,70,9,84,28,45,3

If the user selects the following inputs in the terminal:
Operation 1: max
Operation 2: max
Operation 3: max

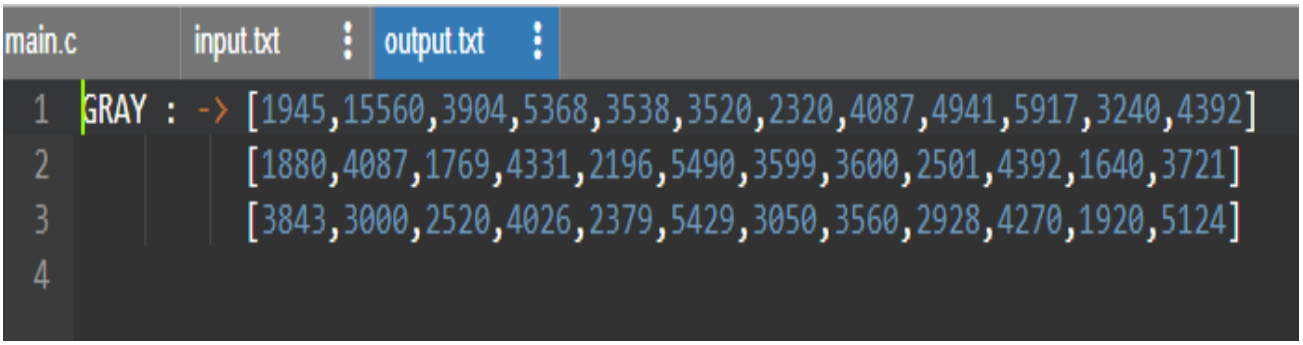
Color mode: gray
Below are example screenshots of the program running and the generated output file.

➞ Terminal execution example:

```
Enter operation 1 (max/avg): max
Enter operation 2 (max/avg): max
Enter operation 3 (max/avg): max
Enter color mode (rgb/bgr/gray): gray
Output written to output.txt


...Program finished with exit code 0
Press ENTER to exit console.
```

➞ output.txt viewed in a text editor:



4. Conclusion

This program successfully fulfills all requirements stated in Assignment 8: stack implementation, user-defined operation selection, chunked processing using max/avg logic, and formatted output generation.